SQL Code

```sql
-- ========================================================================
-- AIRLINE RESERVATION SYSTEM : Sample Data Insertion
-- Student : R Monisha Bangera (2511021240053)
-- Run AFTER airline_reservation_system.sql (tables must already exist).
-- Target  : MySQL 8.0.16+ (uses a recursive CTE to build the seat maps)
--
-- WARNING : Section 0 empties all nine tables first, so the script can be
--           re-run safely (and also wipes the test rows created by
--           data_integrity_tests.sql). All names, e-mails, phone numbers,
--           flight numbers and fares are fictional and for illustration only.
--
-- Expected row counts: Passenger 20 | Airport 12 | Aircraft 8 | Flight 28
--                      Seat 1392 | Booking 50 | Ticket 47 | Payment 51 |
Baggage 43
-- Dates : "today" in the scenario is 24 Oct 2026, so bookings were made in
--         Sep-Oct 2026 and the flights depart in Nov 2026. Times are local.
-- ========================================================================
USE airline_reservation_system;

-- 0. CLEAN START ------------------------------------------------------
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE Baggage;
TRUNCATE TABLE Payment;
TRUNCATE TABLE Ticket;
TRUNCATE TABLE Booking;
TRUNCATE TABLE Seat;
TRUNCATE TABLE Flight;
TRUNCATE TABLE Aircraft;
TRUNCATE TABLE Airport;
TRUNCATE TABLE Passenger;
SET FOREIGN_KEY_CHECKS = 1;

-- 1. PASSENGER (20) ---------------------------------------------------
INSERT INTO Passenger (Passenger_ID, First_Name, Last_Name, Email, Phone,
Date_of_Birth, Gender) VALUES
 ( 1, 'Monisha',   'Bangera',      'monisha.bangera@example.com',
'9000010001', '2005-03-14', 'Female'),
 ( 2, 'Rahul',     'Sharma',       'rahul.sharma@example.com',
'9000010002', '1999-07-21', 'Male'),
 ( 3, 'Priya',     'Nair',         'priya.nair@example.com',
'9000010003', '1992-11-02', 'Female'),
 ( 4, 'Arjun',     'Reddy',        'arjun.reddy@example.com',
'9000010004', '1988-05-30', 'Male'),
 ( 5, 'Ananya',    'Iyer',         'ananya.iyer@example.com',
'9000010005', '2001-09-17', 'Female'),
 ( 6, 'Vikram',    'Singh',        'vikram.singh@example.com',
'9000010006', '1985-01-12', 'Male'),
 ( 7, 'Neha',      'Singh',        'neha.singh@example.com',
'9000010007', '1987-04-08', 'Female'),
```

```sql
 ( 8, 'Mohammed',  'Faizan',       'mohammed.faizan@example.com',
'9000010008', '1995-12-25', 'Male'),
 ( 9, 'Sneha',     'Kulkarni',     'sneha.kulkarni@example.com',
'9000010009', '1998-06-19', 'Female'),
 (10, 'Karthik',   'Subramanian',
'karthik.subramanian@example.com','9000010010','1990-02-28', 'Male'),
 (11, 'Deepa',     'Menon',        'deepa.menon@example.com',
'9000010011', '1979-10-05', 'Female'),
 (12, 'Aditya',    'Verma',        'aditya.verma@example.com',
'9000010012', '1993-08-14', 'Male'),
 (13, 'Pooja',     'Verma',        'pooja.verma@example.com',
'9000010013', '1994-03-22', 'Female'),
 (14, 'Rohan',     'Mehta',        'rohan.mehta@example.com',
'9000010014', '1982-07-07', 'Male'),
 (15, 'Ishita',    'Gupta',        'ishita.gupta@example.com',
'9000010015', '1996-01-30', 'Female'),
 (16, 'Suresh',    'Babu',         'suresh.babu@example.com',
'9000010016', '1975-09-09', 'Male'),
 (17, 'Lakshmi',   'Narayanan',    'lakshmi.narayanan@example.com',
'9000010017', '1968-12-03', 'Female'),
 (18, 'Aarav',     'Patel',        'aarav.patel@example.com',
'9000010018', '1980-11-11', 'Male'),
 (19, 'Fatima',    'Sheikh',       'fatima.sheikh@example.com',
'9000010019', '2000-05-05', 'Female'),
 (20, 'Kiran',     'Rao',          'kiran.rao@example.com',
'9000010020', '1997-10-10', 'Other');

-- 2. AIRPORT (12) ----------------------------------------------------
INSERT INTO Airport (Airport_ID, Airport_Code, Airport_Name, City, Country,
Terminal_Count) VALUES
 ( 1, 'BLR', 'Kempegowda International Airport',                'Bengaluru',
'India',                2),
 ( 2, 'DEL', 'Indira Gandhi International Airport',             'Delhi',
'India',                3),
 ( 3, 'BOM', 'Chhatrapati Shivaji Maharaj International Airport','Mumbai',
'India',                2),
 ( 4, 'MAA', 'Chennai International Airport',                   'Chennai',
'India',                4),
 ( 5, 'HYD', 'Rajiv Gandhi International Airport',              'Hyderabad',
'India',                1),
 ( 6, 'CCU', 'Netaji Subhas Chandra Bose International Airport', 'Kolkata',
'India',                2),
 ( 7, 'GOI', 'Dabolim Airport',                                 'Goa',
'India',                1),
 ( 8, 'COK', 'Cochin International Airport',                    'Kochi',
'India',                3),
 ( 9, 'AMD', 'Sardar Vallabhbhai Patel International Airport',  'Ahmedabad',
'India',                2),
 (10, 'PNQ', 'Pune Airport',                                    'Pune',
'India',                1),
 (11, 'DXB', 'Dubai International Airport',                     'Dubai',
'United Arab Emirates',  3),
 (12, 'SIN', 'Singapore Changi Airport',                        'Singapore',
'Singapore',            4);
```

```sql

-- 3. AIRCRAFT (8) -----------------------------------------------------
-- Total_Seats matches the seat map generated in section 5
-- (A320neo 30 rows x 6 = 180, A321 37 x 6 = 222, 737-800 31 x 6 = 186, ATR 18 x 4 = 72).
INSERT INTO Aircraft (Aircraft_ID, Aircraft_Model, Registration_No, Total_Seats, Aircraft_Status) VALUES
 (1, 'Airbus A320neo',  'VT-IAA', 180, 'Active'),
 (2, 'Airbus A320neo',  'VT-IAB', 180, 'Active'),
 (3, 'Airbus A321',     'VT-IAC', 222, 'Active'),
 (4, 'Boeing 737-800',  'VT-SGA', 186, 'Active'),
 (5, 'Boeing 737-800',  'VT-SGB', 186, 'Maintenance'),   -- grounded: no flights assigned
 (6, 'ATR 72-600',      'VT-ATA',  72, 'Active'),
 (7, 'Airbus A320neo',  'VT-IAD', 180, 'Active'),
 (8, 'Boeing 737-800',  'VT-SGC', 186, 'Active');

-- 4. FLIGHT (28) ------------------------------------------------------
INSERT INTO Flight (Flight_ID, Flight_Number, Aircraft_ID, Departure_Airport_ID, Arrival_Airport_ID,
                    Departure_DateTime, Arrival_DateTime, Flight_Status) VALUES
 (101, 'AI502', 1,  1,  2, '2026-11-01 06:00:00', '2026-11-01 08:50:00', 'Scheduled'),  -- BLR-DEL
 (102, 'AI502', 1,  1,  2, '2026-11-02 06:00:00', '2026-11-02 08:50:00', 'Scheduled'),
 (103, 'AI502', 1,  1,  2, '2026-11-03 06:00:00', '2026-11-03 08:50:00', 'Scheduled'),
 (104, 'AI503', 1,  2,  1, '2026-11-01 10:00:00', '2026-11-01 12:50:00', 'Scheduled'),  -- DEL-BLR
 (105, 'AI503', 1,  2,  1, '2026-11-02 10:00:00', '2026-11-02 12:50:00', 'Scheduled'),
 (106, 'AI503', 1,  2,  1, '2026-11-03 10:00:00', '2026-11-03 12:50:00', 'Scheduled'),
 (107, '6E204', 2,  1,  3, '2026-11-01 09:30:00', '2026-11-01 11:20:00', 'Scheduled'),  -- BLR-BOM
 (108, '6E205', 2,  3,  1, '2026-11-01 12:30:00', '2026-11-01 14:20:00', 'Scheduled'),  -- BOM-BLR
 (109, '6E204', 2,  1,  3, '2026-11-02 09:30:00', '2026-11-02 11:20:00', 'Scheduled'),
 (110, '6E310', 2,  1,  5, '2026-11-01 16:00:00', '2026-11-01 17:20:00', 'Scheduled'),  -- BLR-HYD
 (111, 'UK821', 3,  3,  2, '2026-11-01 07:15:00', '2026-11-01 09:25:00', 'Scheduled'),  -- BOM-DEL
 (112, 'UK822', 3,  2,  3, '2026-11-01 11:00:00', '2026-11-01 13:10:00', 'Scheduled'),  -- DEL-BOM
 (113, 'UK821', 3,  3,  2, '2026-11-02 07:15:00', '2026-11-02 09:25:00', 'Delayed'),
 (114, 'SG145', 4,  2,  6, '2026-11-01 08:00:00', '2026-11-01 10:15:00', 'Scheduled'),  -- DEL-CCU
 (115, 'SG146', 4,  6,  2, '2026-11-01 11:30:00', '2026-11-01 13:45:00', 'Scheduled'),  -- CCU-DEL
 (116, 'SG145', 4,  2,  6, '2026-11-02 08:00:00', '2026-11-02 10:15:00', 'Scheduled'),
```

```sql
 (117, 'IX771', 6,  1,  8, '2026-11-01 07:30:00', '2026-11-01 08:40:00', 'Scheduled'),  -- BLR-COK
 (118, 'IX772', 6,  8,  1, '2026-11-01 09:30:00', '2026-11-01 10:40:00', 'Scheduled'),  -- COK-BLR
 (119, 'IX771', 6,  1,  8, '2026-11-02 07:30:00', '2026-11-02 08:40:00', 'Scheduled'),
 (120, 'AI934', 7,  1, 11, '2026-11-01 02:30:00', '2026-11-01 05:10:00', 'Scheduled'),  -- BLR-DXB
 (121, 'AI935', 7, 11,  1, '2026-11-01 07:00:00', '2026-11-01 12:10:00', 'Delayed'),    -- DXB-BLR
 (122, 'AI934', 7,  1, 11, '2026-11-03 02:30:00', '2026-11-03 05:10:00', 'Scheduled'),
 (123, '6E411', 8,  4,  1, '2026-11-01 13:00:00', '2026-11-01 14:05:00', 'Scheduled'),  -- MAA-BLR
 (124, '6E412', 8,  1,  4, '2026-11-01 15:00:00', '2026-11-01 16:05:00', 'Scheduled'),  -- BLR-MAA
 (125, '6E640', 8,  1,  7, '2026-11-02 11:00:00', '2026-11-02 12:15:00', 'Scheduled'),  -- BLR-GOI
 (126, '6E640', 8,  1,  7, '2026-11-03 11:00:00', '2026-11-03 12:15:00', 'Cancelled'),
 (127, 'UK955', 3,  2,  5, '2026-11-02 14:00:00', '2026-11-02 16:20:00', 'Scheduled'),  -- DEL-HYD
 (128, 'AI346', 2,  1, 12, '2026-11-03 23:55:00', '2026-11-04 06:50:00', 'Scheduled');  -- BLR-SIN

-- 5. SEAT (1392) : full seat map for every aircraft, generated in one statement
--    Letters A-F (A-D on the ATR). Classes: A321 rows 1-2 First, 3-6 Business;
--    A320neo rows 1-3 Business; 737-800 rows 1-2 Business; ATR all Economy.
INSERT INTO Seat (Aircraft_ID, Seat_Number, Seat_Class, Seat_Status)
WITH RECURSIVE seat_rows (n) AS (
    SELECT 1 UNION ALL SELECT n + 1 FROM seat_rows WHERE n < 37
),
seat_letters (idx, letter) AS (
    SELECT 1, 'A' UNION ALL SELECT 2, 'B' UNION ALL SELECT 3, 'C' UNION ALL
    SELECT 4, 'D' UNION ALL SELECT 5, 'E' UNION ALL SELECT 6, 'F'
)
SELECT a.Aircraft_ID,
       CONCAT(r.n, l.letter),
       CASE
         WHEN a.Aircraft_Model LIKE 'ATR%'            THEN 'Economy'
         WHEN a.Aircraft_Model LIKE 'Airbus A321%'    THEN CASE WHEN r.n <= 2 THEN 'First'
                                                                WHEN r.n <= 6 THEN 'Business'
                                                                ELSE 'Economy' END
         WHEN a.Aircraft_Model LIKE 'Airbus A320%'    THEN CASE WHEN r.n <= 3 THEN 'Business'
                                                                ELSE 'Economy' END
         ELSE                                              CASE WHEN r.n <= 2 THEN 'Business'
```

```sql
                                                    ELSE 'Economy'
END
       END,
       'Available'
FROM Aircraft a
JOIN seat_rows r
JOIN seat_letters l
WHERE l.idx <= IF(a.Aircraft_Model LIKE 'ATR%', 4, 6)
  AND r.n   <= a.Total_Seats / IF(a.Aircraft_Model LIKE 'ATR%', 4, 6)
ORDER BY a.Aircraft_ID, r.n, l.idx;
-- Seat_Status stays 'Available' for all seats: a seat is booked per FLIGHT, so
-- availability is derived from the Booking table (see query C4 below).

-- 6. BOOKING (50) -----------------------------------------------------
-- Bookings are loaded through a staging table that holds the seat NUMBER (e.g. 12A).
-- The INSERT ... SELECT finds the Seat_ID on the SAME aircraft as the flight, so a
-- booking can never point to a seat from a different aircraft.
CREATE TEMPORARY TABLE stg_booking (
    Booking_ID     INT,
    Passenger_ID   INT,
    Flight_ID      INT,
    Seat_Number    VARCHAR(10),
    Booking_Date   DATE,
    Booking_Status VARCHAR(20),
    Fare           DECIMAL(10,2)
);

INSERT INTO stg_booking VALUES
 (5001,  1, 101, '12A', '2026-10-10', 'Confirmed',  5500.00),
 (5002,  2, 101, '12B', '2026-10-10', 'Confirmed',  5500.00),
 (5003,  3, 101, '2A',  '2026-10-05', 'Confirmed', 13800.00),   -- Business
 (5004,  4, 101, '18C', '2026-10-15', 'Confirmed',  5200.00),
 (5005,  5, 101, '18D', '2026-10-18', 'Cancelled',  5200.00),
 (5006,  6, 102, '9A',  '2026-10-20', 'Confirmed',  6100.00),
 (5007,  7, 102, '9B',  '2026-10-20', 'Confirmed',  6100.00),
 (5008,  8, 102, '22E', '2026-10-21', 'Pending',    5900.00),
 (5009,  9, 103, '3F',  '2026-10-22', 'Confirmed', 14500.00),   -- Business
 (5010, 10, 103, '15A', '2026-10-22', 'Confirmed',  6400.00),
 (5011,  1, 104, '11A', '2026-10-10', 'Confirmed',  5800.00),   -- return of 5001
 (5012,  2, 104, '11B', '2026-10-10', 'Confirmed',  5800.00),   -- return of 5002
 (5013,  4, 104, '20C', '2026-10-15', 'Confirmed',  5600.00),
 (5014, 11, 105, '7D',  '2026-10-19', 'Confirmed',  6000.00),
 (5015, 12, 107, '5A',  '2026-10-08', 'Confirmed',  4800.00),
 (5016, 13, 107, '5B',  '2026-10-08', 'Confirmed',  4800.00),
 (5017, 14, 107, '1C',  '2026-10-11', 'Confirmed', 11900.00),   -- Business
 (5018, 15, 107, '25F', '2026-10-17', 'Cancelled',  4500.00),
 (5019, 12, 108, '6A',  '2026-10-08', 'Confirmed',  4700.00),   -- return of 5015
```

```sql
 (5020, 13, 108, '6B',  '2026-10-08', 'Confirmed',  4700.00),   -- return of 5016
 (5021, 16, 110, '10C', '2026-10-20', 'Confirmed',  3200.00),
 (5022, 17, 110, '10D', '2026-10-20', 'Pending',    3200.00),
 (5023, 18, 111, '1A',  '2026-10-01', 'Confirmed', 21500.00),   -- First
 (5024, 19, 111, '4C',  '2026-10-03', 'Confirmed', 12800.00),   -- Business
 (5025, 20, 111, '15B', '2026-10-14', 'Confirmed',  5400.00),
 (5026,  3, 111, '15C', '2026-10-14', 'Confirmed',  5400.00),
 (5027, 18, 112, '1A',  '2026-10-01', 'Confirmed', 21800.00),   -- First, return of 5023
 (5028, 20, 112, '30E', '2026-10-14', 'Confirmed',  5300.00),
 (5029,  5, 113, '12F', '2026-10-16', 'Confirmed',  5600.00),   -- delayed flight
 (5030,  6, 114, '8A',  '2026-10-13', 'Confirmed',  5900.00),
 (5031,  7, 114, '8B',  '2026-10-13', 'Confirmed',  5900.00),
 (5032,  8, 114, '1D',  '2026-10-09', 'Confirmed', 12400.00),   -- Business
 (5033,  6, 115, '9A',  '2026-10-13', 'Confirmed',  6200.00),   -- return of 5030
 (5034,  7, 115, '9B',  '2026-10-13', 'Confirmed',  6200.00),   -- return of 5031
 (5035,  9, 117, '3A',  '2026-10-18', 'Confirmed',  3400.00),
 (5036, 10, 117, '3B',  '2026-10-18', 'Confirmed',  3400.00),
 (5037, 11, 117, '10D', '2026-10-21', 'Pending',    3300.00),
 (5038, 14, 120, '2A',  '2026-09-28', 'Confirmed', 21500.00),   -- Business
 (5039, 15, 120, '20A', '2026-10-02', 'Confirmed', 14800.00),
 (5040, 16, 120, '20B', '2026-10-02', 'Confirmed', 14800.00),
 (5041, 14, 121, '2A',  '2026-09-28', 'Confirmed', 22000.00),   -- return of 5038
 (5042, 15, 121, '21A', '2026-10-02', 'Confirmed', 15200.00),   -- return of 5039
 (5043, 17, 124, '12A', '2026-10-19', 'Confirmed',  2900.00),
 (5044, 19, 124, '12B', '2026-10-19', 'Confirmed',  2900.00),
 (5045, 13, 126, '6A',  '2026-10-05', 'Cancelled',  4100.00),   -- flight 126 was cancelled
 (5046, 12, 126, '6B',  '2026-10-05', 'Cancelled',  4100.00),
 (5047, 18, 128, '3A',  '2026-10-17', 'Confirmed', 38500.00),   -- Business
 (5048,  2, 128, '17C', '2026-10-18', 'Confirmed', 17800.00),
 (5049,  3, 125, '9C',  '2026-10-20', 'Confirmed',  4300.00),
 (5050,  4, 125, '9D',  '2026-10-20', 'Confirmed',  4300.00);

INSERT INTO Booking (Booking_ID, Passenger_ID, Flight_ID, Seat_ID, Booking_Date, Booking_Status, Fare)
SELECT s.Booking_ID, s.Passenger_ID, s.Flight_ID, st.Seat_ID, s.Booking_Date, s.Booking_Status, s.Fare
FROM   stg_booking s
JOIN   Flight f  ON f.Flight_ID   = s.Flight_ID
JOIN   Seat   st ON st.Aircraft_ID = f.Aircraft_ID AND st.Seat_Number = s.Seat_Number
ORDER  BY s.Booking_ID;

-- Both numbers must be 50; if Inserted is lower, a seat number does not exist on that aircraft.
SELECT (SELECT COUNT(*) FROM stg_booking) AS Staged,
       (SELECT COUNT(*) FROM Booking)     AS Inserted;
```

```sql

DROP TEMPORARY TABLE stg_booking;

-- 7. BAGGAGE (43) : one record per confirmed booking --------------------
INSERT INTO Baggage (Booking_ID, Baggage_Count, Total_Weight,
Excess_Baggage_Fee) VALUES
 (5001, 1, 14.50,    0.00), (5002, 2, 26.00,  800.00), (5003, 2, 32.00,
0.00),
 (5004, 1, 18.20,  500.00), (5006, 1, 12.00,    0.00), (5007, 1, 15.00,
0.00),
 (5009, 2, 30.50,    0.00), (5010, 1, 16.40,  300.00), (5011, 1, 14.00,
0.00),
 (5012, 2, 24.60,  700.00), (5013, 1, 13.80,    0.00), (5014, 0,  0.00,
0.00),
 (5015, 1, 15.00,    0.00), (5016, 1, 17.50,  400.00), (5017, 2, 28.00,
0.00),
 (5019, 1, 15.00,    0.00), (5020, 1, 19.00,  600.00), (5021, 1, 10.00,
0.00),
 (5023, 3, 45.00,    0.00), (5024, 2, 29.50,    0.00), (5025, 1, 14.20,
0.00),
 (5026, 1, 21.00,  900.00), (5027, 3, 46.50,    0.00), (5028, 1, 15.00,
0.00),
 (5029, 1, 13.00,    0.00), (5030, 1, 15.00,    0.00), (5031, 2, 27.30,
1000.00),
 (5032, 1, 20.00,    0.00), (5033, 1, 15.00,    0.00), (5034, 2, 25.00,
800.00),
 (5035, 1, 12.50,    0.00), (5036, 1, 16.00,  350.00), (5038, 2, 35.00,
0.00),
 (5039, 2, 40.00, 1500.00), (5040, 1, 23.00,    0.00), (5041, 2, 36.00,
0.00),
 (5042, 2, 44.00, 2500.00), (5043, 1, 14.00,    0.00), (5044, 1, 15.00,
0.00),
 (5047, 2, 34.00,    0.00), (5048, 2, 38.00, 1800.00), (5049, 1, 15.00,
0.00),
 (5050, 1, 22.00,  700.00);

-- 8. TICKET (47) : issued for confirmed bookings; cancelled bookings keep a
cancelled ticket
INSERT INTO Ticket (Booking_ID, Ticket_Number, Issue_Date, Ticket_Status)
SELECT b.Booking_ID,
       CONCAT('TKT-', f.Flight_Number, '-', LPAD(b.Booking_ID, 5, '0')),
       b.Booking_Date,
       IF(b.Booking_Status = 'Confirmed', 'Valid', 'Cancelled')
FROM   Booking b
JOIN   Flight  f ON f.Flight_ID = b.Flight_ID
WHERE  b.Booking_Status IN ('Confirmed', 'Cancelled')
ORDER  BY b.Booking_ID;

-- 9. PAYMENT (51) : amount = fare + excess baggage fee; pending bookings have
a pending payment
INSERT INTO Payment (Booking_ID, Payment_Date, Payment_Method, Amount,
Payment_Status, Transaction_ID)
SELECT b.Booking_ID,
       b.Booking_Date,
```

```sql
       ELT(MOD(b.Booking_ID, 3) + 1, 'UPI', 'Card', 'Net Banking'),
       b.Fare + COALESCE(g.Excess_Baggage_Fee, 0),
       IF(b.Booking_Status = 'Pending', 'Pending', 'Successful'),
       CONCAT('TXN', DATE_FORMAT(b.Booking_Date, '%Y%m%d'), LPAD(b.Booking_ID, 5, '0'))
FROM   Booking b
LEFT   JOIN Baggage g ON g.Booking_ID = b.Booking_ID
ORDER  BY b.Booking_ID;

-- A failed first attempt for booking 5004 (a retry succeeded above): a booking can have several payments
INSERT INTO Payment (Booking_ID, Payment_Date, Payment_Method, Amount, Payment_Status, Transaction_ID)
VALUES (5004, '2026-10-15', 'Card', 5200.00, 'Failed', 'TXN20261015F5004');

-- ======================================================================
-- VERIFICATION
-- ======================================================================

-- C1  Row count of every table
SELECT 'Passenger' AS Tbl, COUNT(*) AS Row_Count FROM Passenger UNION ALL
SELECT 'Airport',   COUNT(*) FROM Airport  UNION ALL
SELECT 'Aircraft',  COUNT(*) FROM Aircraft UNION ALL
SELECT 'Flight',    COUNT(*) FROM Flight   UNION ALL
SELECT 'Seat',      COUNT(*) FROM Seat     UNION ALL
SELECT 'Booking',   COUNT(*) FROM Booking  UNION ALL
SELECT 'Ticket',    COUNT(*) FROM Ticket   UNION ALL
SELECT 'Payment',   COUNT(*) FROM Payment  UNION ALL
SELECT 'Baggage',   COUNT(*) FROM Baggage;

-- C2  Seat map per aircraft equals Aircraft.Total_Seats (Difference must be 0 for every row)
SELECT a.Aircraft_ID, a.Aircraft_Model, a.Total_Seats, COUNT(s.Seat_ID) AS Seats_Generated,
       a.Total_Seats - COUNT(s.Seat_ID) AS Difference
FROM   Aircraft a LEFT JOIN Seat s ON s.Aircraft_ID = a.Aircraft_ID
GROUP  BY a.Aircraft_ID, a.Aircraft_Model, a.Total_Seats;

-- C3  Bookings and revenue by status
SELECT b.Booking_Status, COUNT(*) AS Bookings, SUM(b.Fare) AS Total_Fare
FROM   Booking b GROUP BY b.Booking_Status;

-- C4  Seat availability per flight, derived from bookings (cancelled bookings free the seat in practice)
SELECT f.Flight_ID, f.Flight_Number, dep.Airport_Code AS Origin, arr.Airport_Code AS Destination,
       f.Departure_DateTime, f.Flight_Status, a.Total_Seats,
       COUNT(b.Booking_ID)                  AS Active_Bookings,
       a.Total_Seats - COUNT(b.Booking_ID)  AS Seats_Left
FROM   Flight f
JOIN   Aircraft a   ON a.Aircraft_ID  = f.Aircraft_ID
JOIN   Airport dep  ON dep.Airport_ID = f.Departure_Airport_ID
JOIN   Airport arr  ON arr.Airport_ID = f.Arrival_Airport_ID
```

```sql
LEFT   JOIN Booking b ON b.Flight_ID = f.Flight_ID AND b.Booking_Status <> 'Cancelled'
GROUP  BY f.Flight_ID, f.Flight_Number, dep.Airport_Code, arr.Airport_Code,
          f.Departure_DateTime, f.Flight_Status, a.Total_Seats
ORDER  BY f.Departure_DateTime, f.Flight_ID;

-- C5  Full itinerary of passenger 1 (round trip BLR-DEL-BLR) across all related tables
SELECT CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger,
       b.Booking_ID, f.Flight_Number, dep.Airport_Code AS From_Airport, arr.Airport_Code AS To_Airport,
       f.Departure_DateTime, s.Seat_Number, s.Seat_Class, b.Booking_Status,
       t.Ticket_Number, t.Ticket_Status, pay.Payment_Method, pay.Amount, pay.Payment_Status,
       g.Baggage_Count, g.Total_Weight
FROM   Passenger p
JOIN   Booking  b   ON b.Passenger_ID = p.Passenger_ID
JOIN   Flight   f   ON f.Flight_ID    = b.Flight_ID
JOIN   Airport  dep ON dep.Airport_ID = f.Departure_Airport_ID
JOIN   Airport  arr ON arr.Airport_ID = f.Arrival_Airport_ID
JOIN   Seat     s   ON s.Seat_ID      = b.Seat_ID
LEFT   JOIN Ticket  t   ON t.Booking_ID   = b.Booking_ID
LEFT   JOIN Payment pay ON pay.Booking_ID = b.Booking_ID AND pay.Payment_Status = 'Successful'
LEFT   JOIN Baggage g   ON g.Booking_ID   = b.Booking_ID
WHERE  p.Passenger_ID = 1
ORDER  BY f.Departure_DateTime;

-- C6  Seat/aircraft consistency: every booked seat belongs to the aircraft flying that flight (must return 0 rows)
SELECT b.Booking_ID, f.Aircraft_ID AS Flight_Aircraft, s.Aircraft_ID AS Seat_Aircraft
FROM   Booking b
JOIN   Flight f ON f.Flight_ID = b.Flight_ID
JOIN   Seat   s ON s.Seat_ID   = b.Seat_ID
WHERE  f.Aircraft_ID <> s.Aircraft_ID;

-- C7  Payment totals match fare + baggage fee for successful payments (must return 0 rows)
SELECT b.Booking_ID, b.Fare, COALESCE(g.Excess_Baggage_Fee, 0) AS Baggage_Fee, p.Amount
FROM   Booking b
JOIN   Payment p      ON p.Booking_ID = b.Booking_ID AND p.Payment_Status = 'Successful'
LEFT   JOIN Baggage g ON g.Booking_ID = b.Booking_ID
WHERE  p.Amount <> b.Fare + COALESCE(g.Excess_Baggage_Fee, 0);

-- C8  Orphan checks: every query must return 0 rows
SELECT Booking_ID FROM Booking WHERE Passenger_ID NOT IN (SELECT Passenger_ID FROM Passenger);
SELECT Booking_ID FROM Booking WHERE Flight_ID    NOT IN (SELECT Flight_ID FROM Flight);
SELECT Booking_ID FROM Booking WHERE Seat_ID      NOT IN (SELECT Seat_ID FROM Seat);
```

```sql
SELECT Ticket_ID  FROM Ticket  WHERE Booking_ID   NOT IN (SELECT Booking_ID FROM Booking);
SELECT Payment_ID FROM Payment WHERE Booking_ID   NOT IN (SELECT Booking_ID FROM Booking);
SELECT Baggage_ID FROM Baggage WHERE Booking_ID   NOT IN (SELECT Booking_ID FROM Booking);
SELECT Flight_ID  FROM Flight  WHERE Aircraft_ID  NOT IN (SELECT Aircraft_ID FROM Aircraft);

-- C9  Busiest routes by confirmed bookings and revenue collected
SELECT CONCAT(dep.Airport_Code, ' -> ', arr.Airport_Code) AS Route,
       COUNT(b.Booking_ID) AS Confirmed_Bookings, SUM(b.Fare) AS Fare_Total
FROM   Booking b
JOIN   Flight  f   ON f.Flight_ID    = b.Flight_ID
JOIN   Airport dep ON dep.Airport_ID = f.Departure_Airport_ID
JOIN   Airport arr ON arr.Airport_ID = f.Arrival_Airport_ID
WHERE  b.Booking_Status = 'Confirmed'
GROUP  BY dep.Airport_Code, arr.Airport_Code
ORDER  BY Confirmed_Bookings DESC, Fare_Total DESC;

```